

WEB TECHNOLOGIES LAB REPORT

Semester Project: Task Hub

A full-stack task and project management web application

Course	Web Technologies
Project Title	Task Hub
Frontend	Angular, TypeScript, HTML, CSS
Backend	ASP.NET Core Web API, C#
Database	SQL relational database scripts included
Submitted By	Hafiz Muhammad Ammar Ali (2502123) Saad Khalid (2502137) Marij Khan (2502151)
Submission Date	June 01, 2026
Institute	Air University

Table of Contents

1. Introduction
2. Problem Statement
3. Project Objectives
4. Technology Stack
5. System Architecture
6. Database Design
7. Backend API Documentation
8. Frontend Module Description
9. JavaScript and DOM Demonstration
10. UI Screenshots
11. Setup and Deployment Steps
12. Testing and Validation
13. Requirements Checklist
14. Video Demo Plan
15. Conclusion

1. Introduction

Task Hub is a full-stack web application designed for managing projects, tasks, categories, comments, users, and activity logs. The system supports authenticated users, project-based task organization, dashboard statistics, and CRUD operations through a RESTful backend API. This report is prepared according to the Web Technologies semester project requirements, which require Angular for the frontend, ASP.NET Core Web API for the backend, and a SQL Server or MongoDB database.

2. Problem Statement

Many student and team projects fail to stay organized because tasks, deadlines, responsibilities, and updates are scattered across messages or informal notes. Task Hub solves this problem by providing a centralized web platform where users can create projects, assign and track tasks, categorize work, add comments, and monitor progress from a dashboard.

3. Project Objectives

- Provide a responsive web interface for project and task management.
- Implement secure login and signup using JWT authentication.
- Connect the Angular frontend with ASP.NET Core backend APIs using HTTP Client.
- Store real project, task, user, category, comment, and activity data in a relational database.
- Provide complete CRUD operations for the main entities.
- Demonstrate dynamic rendering, form validation, routing, and database-driven content.

4. Technology Stack

Layer	Technology	Purpose
Frontend	Angular + TypeScript + HTML + CSS	Dynamic pages, routing, forms, services, and responsive UI
Backend	ASP.NET Core Web API (C#)	REST APIs, authentication, validation, and business logic
Database	SQL relational database	Persistent storage for users, projects, tasks, categories, comments, and logs
Authentication	JWT	Token-based secure access for protected routes and APIs
Tools	VS Code, GitHub, Swagger/Postman	Development, version control, and API testing

5. System Architecture

The project follows a three-tier architecture. The Angular frontend communicates with the ASP.NET Core Web API using HTTP requests. The backend applies authentication, validation, and business rules before reading or writing records in the database. The database stores related records for users, projects, tasks, categories, comments, and activity logs.

System Architecture Diagram Placeholder
Angular Frontend -> ASP.NET Core Web API -> SQL Database

Swagger
OPENAPI SPECIFICATION

Select a definition TaskHub.Api v1

TaskHub.Api 1.0 OAS 3.0
https://localhost:7162/swagger/v1/swagger.json

Auth

POST /api/auth/signup

POST /api/auth/login

Parameters

No parameters

Request body

application/json

```
{
  "email": "admin@taskhub.local",
  "password": "Admin123!"
}
```

Execute Clear

Responses

Component	Responsibility	Main Files/Modules
Frontend	User interface, routing, reactive forms, dashboard, CRUD pages	/frontend Angular application
Backend	API controllers, authentication, DTOs, services, database context	/backend ASP.NET Core project
Database	Schema, seed data, and relational tables	/database/schema.sql, /database/seed.sql
Demo/Report	Submission material for demo and documentation	/demo and /report folders

6. Database Design

The database design is based on related entities. A user can own or participate in projects. Projects contain tasks. Tasks can have categories and comments. Activity logs store important user and task actions for audit and dashboard display.

Detected backend models in the submitted archive: ActivityLog, AppUser, Category, Project, ProjectTask, TaskComment.

Table/Collection	Main Fields	Relationship/Purpose
AppUsers / Users	Id, full name, email, password hash, role	Stores registered users and supports authentication
Projects	Id, title, description, owner/user id, dates	A user can create and manage multiple projects
ProjectTasks / Tasks	Id, title, description, status, priority, due date, project id	Tasks belong to projects and track work progress
Categories	Id, name, color/description	Used to classify tasks or projects
TaskComments	Id, task id, user id, comment text, created date	Allows discussion and updates on tasks

ActivityLogs	Id, user id, action, entity, timestamp	Stores activity history for dashboard and tracking

7. Backend API Documentation

The backend is designed as a RESTful ASP.NET Core Web API. The following endpoints should be verified in Swagger or Postman before final submission.

Detected controllers in the submitted archive: ApiControllerBase, AuthController, CategoriesController, CommentsController, DashboardController, ProjectsController, TasksController, UsersController.

Controller	Method	Endpoint	Purpose	Expected Response
Auth	POST	/api/auth/register	Create a new user account	201 Created or success DTO
Auth	POST	/api/auth/login	Validate user credentials and return JWT token	200 OK with token
Projects	GET	/api/projects	Fetch user projects	200 OK with project list
Projects	POST	/api/projects	Create a new project	201 Created
Projects	PUT	/api/projects/{id}	Update project information	200 OK / 204 No Content
Projects	DELETE	/api/projects/{id}	Delete a project	204 No Content
Tasks	GET	/api/tasks	Fetch tasks with project/category data	200 OK with task list
Tasks	POST	/api/tasks	Create a new task	201 Created
Tasks	PUT	/api/tasks/{id}	Update task status/details	200 OK / 204 No Content
Tasks	DELETE	/api/tasks/{id}	Delete a task	204 No Content
Categories	GET/POST/PUT/DELETE	/api/categories	Manage task categories	Standard REST responses
Comments	GET/POST/DELETE	/api/comments	View and add task comments	Standard REST responses
Dashboard	GET	/api/dashboard	Return counts and recent activity for dashboard	200 OK with summary data
Users	GET/PUT/DELETE	/api/users	Admin/user management operations	Standard REST responses

8. Frontend Module Description

The Angular frontend should contain component-based pages, services for API communication, routing for navigation, and reactive forms with validation. The application must not be plain static HTML; dynamic content should be fetched from backend APIs and rendered without page reloads.

Page/Module	Required Feature	Implementation Notes
Home/Landing	Hero section, animations, dynamic content	Introduce Task Hub and show main actions
Authentication	Login and Signup with validation	Use Angular Reactive Forms and connect with Auth API
Dashboard	User-specific database data	Show counts, recent tasks, projects, and activity
Main Feature Page	Core task/project CRUD	Create, update, delete, and filter tasks/projects
Detail/Profile Page	Dynamic routing	Route by id for project/task/profile details
Admin Panel	Manage users/data	Admin CRUD operations for users or records

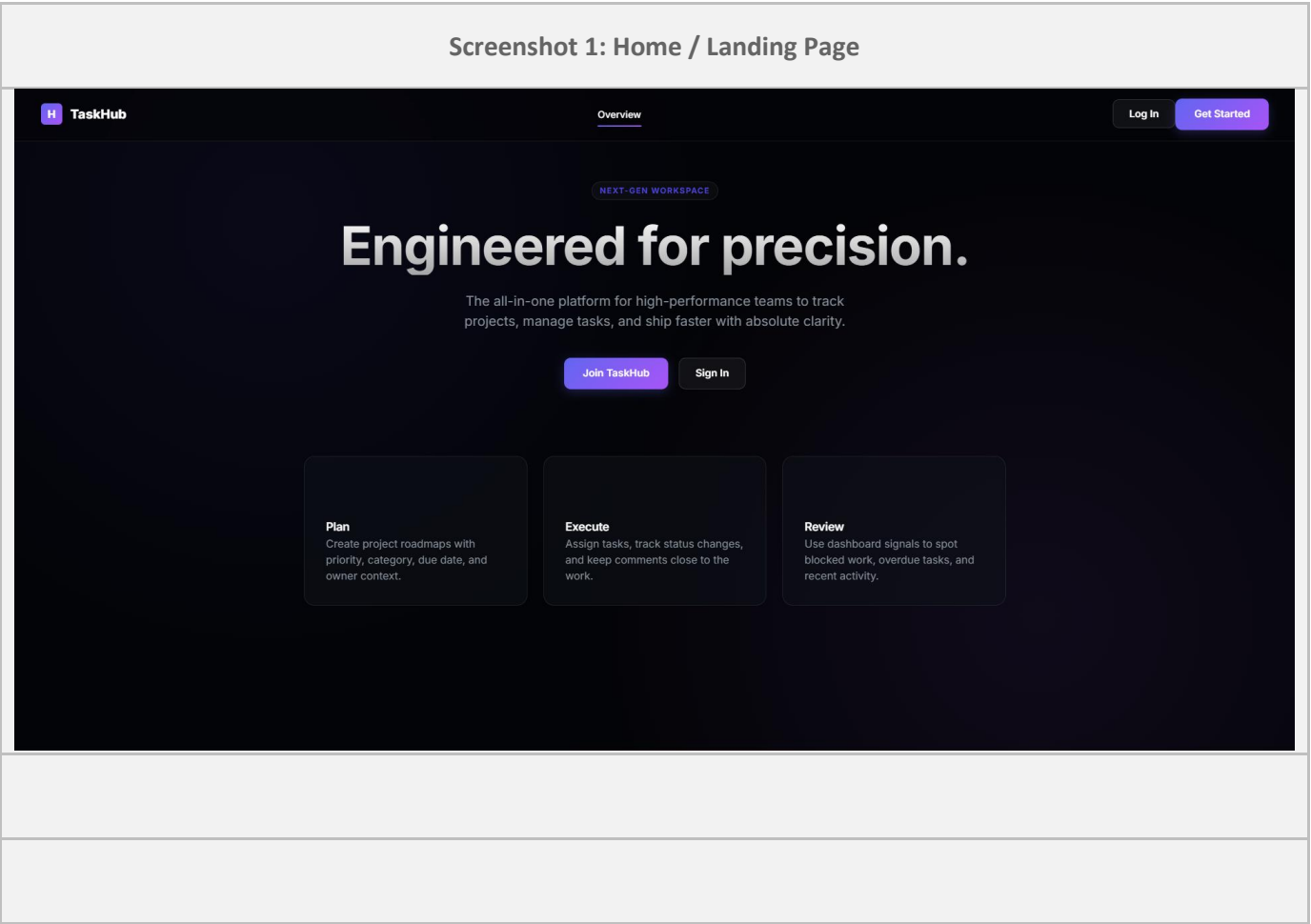
9. JavaScript and DOM Demonstration

The project requirements mention that at least one vanilla JavaScript DOM feature must be demonstrated. In Task Hub, this can be shown through a small real-time UI behavior such as updating a task counter, toggling a notification panel, filtering visible tasks, or showing a live character count without refreshing the page.

DOM Requirement	Suggested Demonstration in Project
Event listeners	Click/filter/change event on task search, sidebar toggle, or theme switch.
Form handling	Live validation feedback or character counter.
Dynamic content rendering	Update visible task cards/counts instantly without page reload.
Real-time UI updates	Show status badge changes immediately after a task status update.

10. UI Screenshots

Paste screenshots of all completed pages in the placeholders below. Each screenshot should show real UI and, where possible, real data from the database.



Screenshot 2: Login Page with Validation

TaskHub Overview Log In Get Started

IDENTITY PROTOCOL

Secure Entry

Authorize your session to access your private workspace.

Email Identifier
name@company.com

Access Key

Authenticate

First time here?
[Request access](#)

localhost:4200/signup

Screenshot 3: Signup Page with Validation

TaskHub Overview Log In Get Started

IDENTITY PROTOCOL

Create Workspace

Join high-performance teams managing complexity with absolute precision.

Display Name
e.g. Alex Rivera

Email Identifier
name@company.com

Access Key

Initialize Account

Already configured?
[Sign in](#)

Screenshot 4: Dashboard with Database Data

H

TaskHub

Overview

Workspace

Projects

Control

Admin User

Sign Out

WORKSPACE

Good day, Admin User

New Project

Total projects2
Active projects1
Open tasks3
Completed tasks0

Priority Projects

Partner Onboarding Playbook

Operations

Planning

Client Portal Refresh

Product

Active

Recent Activity

Alex Reed Added onboarding milestones for operations review.
6:47 PM

Maya Khan Created project roadmap and assigned first tasks.
6:47 PM

My Deliverables

In Progress

Ship dashboard API

Client Portal Refresh

Review

Design activity filters

Client Portal Refresh

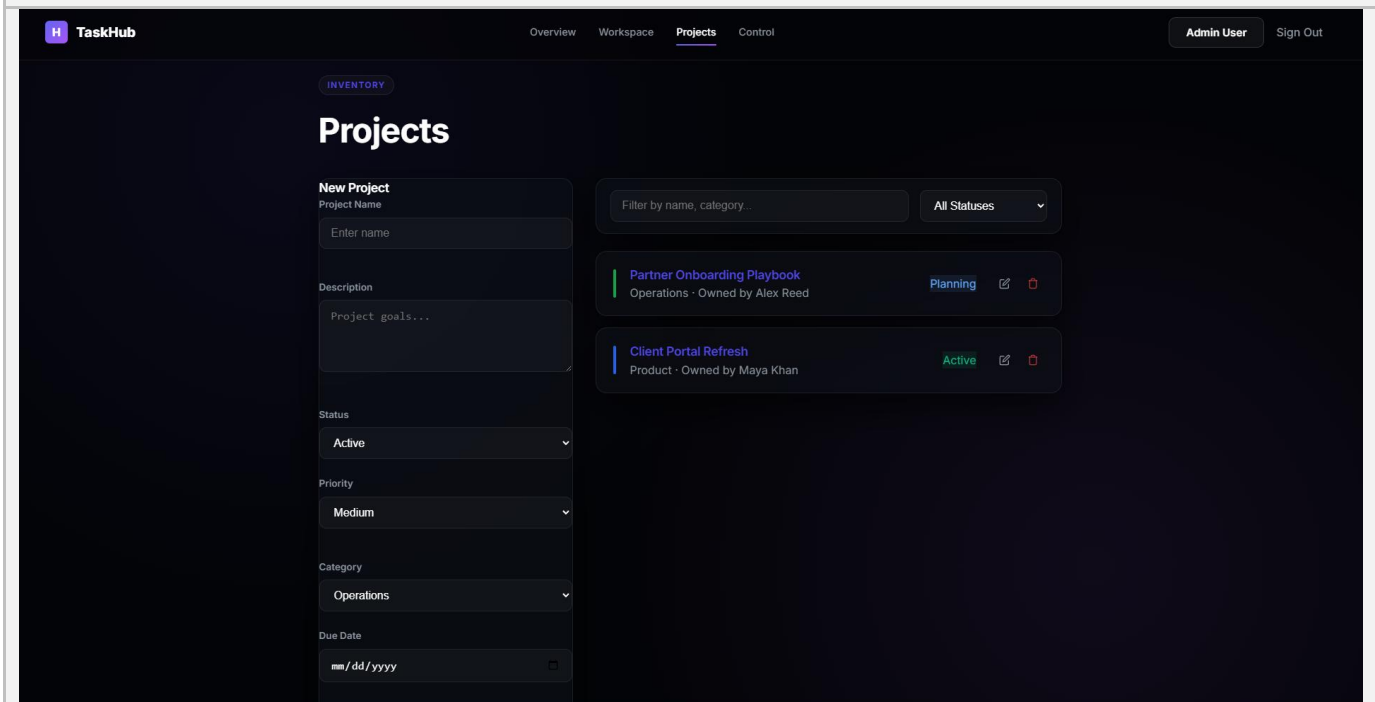
ToDo

Draft kickoff checklist

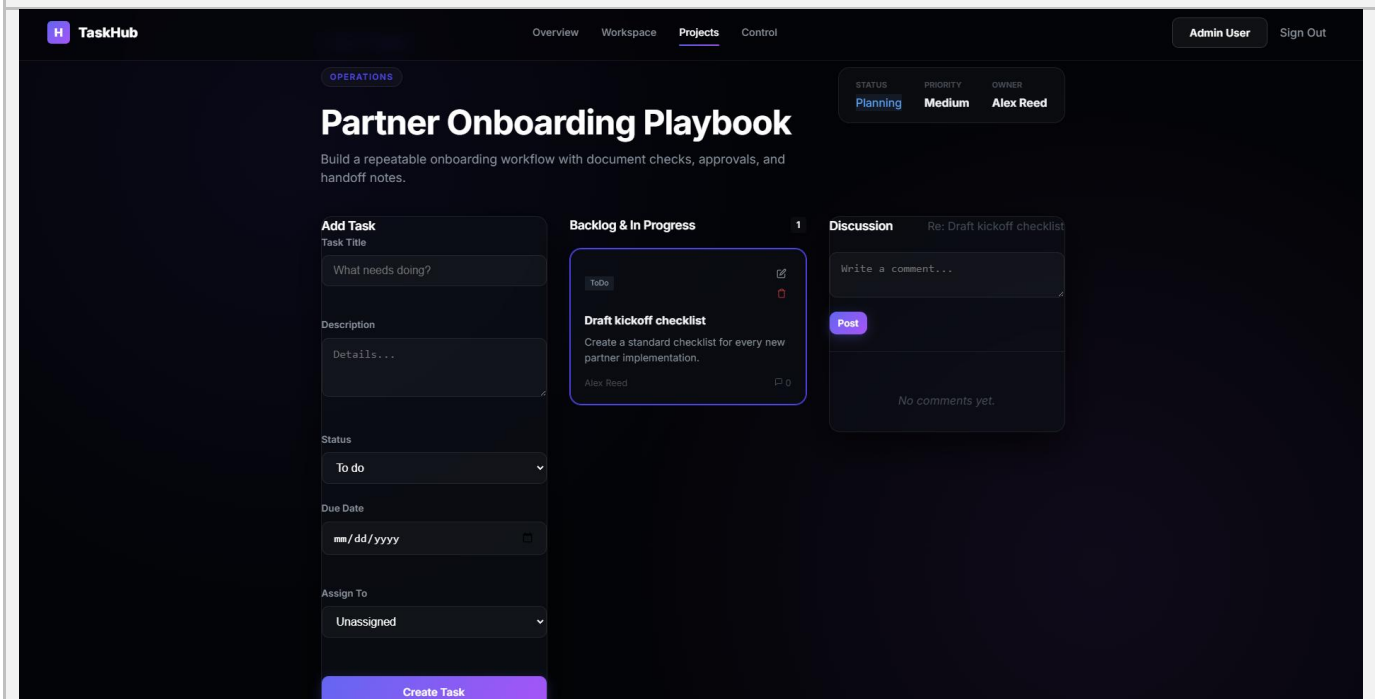
Partner Onboarding Playbook

Focus Notes

Screenshot 5: Projects / Main Feature Page



Screenshot 6: Task Detail or Profile Page with Dynamic Route



Screenshot 7: Admin Panel / CRUD Management

TaskHub

OverviewWorkspaceProjectsControl

Admin UserSign Out

ADMINISTRATION

Control Center

Provision Account

Total users3
3 active accounts
Administrators1
2 standard users
Categories3
Workspace taxonomy
Tracked projects2
Across all categories

User DirectoryTaxonomy

MEMBER	ACCESS ROLE	STATUS	ACTIONS
Admin User admin@taskhub.local	Admin	Active	Edit Delete
Alex Reed alex@taskhub.local	User	Active	Edit Delete
Maya Khan maya@taskhub.local	User	Active	Edit Delete

New Account

Full Name

Email Address

Role

Standard User

☒ Is Active

Screenshot 8: Swagger or Postman API Test

Swagger
powered by SMARTBEAR

Select a definition TaskHub.Api v1

TaskHub.Api 1.0 OAS 3.0

<https://localhost:7162/swagger/v1/swagger.json>

Auth

POST /api/auth/signup

POST /api/auth/login

Parameters Cancel Reset

No parameters

Request body application/json

```
{  "email": "admin@taskhub.local",  "password": "Admin123!"}
```

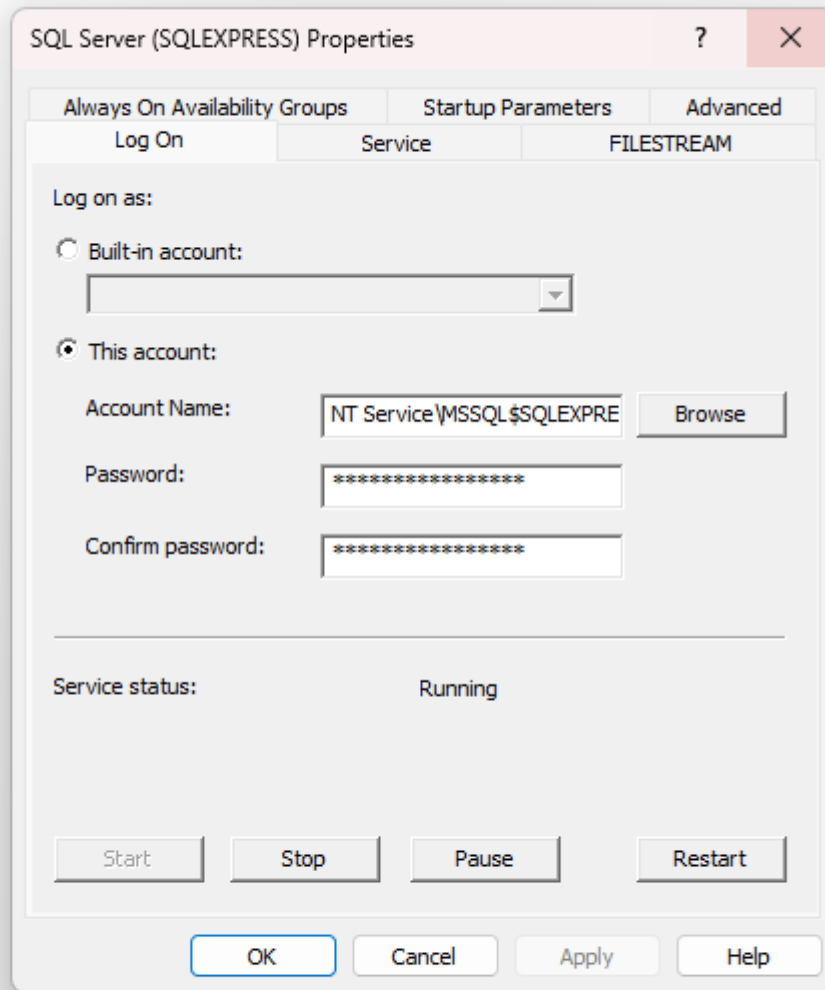
Execute Clear

Responses

Snipping Tool

Screenshot copied to clipboard
Automatically saved to screenshots folder.
Markup and share

Screenshot 9: SQL Server



11. Setup and Deployment Steps

11.1 Local Setup

1. Clone or download the GitHub repository.

- Open the project folder in Visual Studio Code.
- Open the backend folder and run: dotnet restore.
- Configure the database connection string in appsettings.json.
- Run the backend API using: dotnet run.
- Open the frontend folder and run: npm install.
- Start the Angular frontend using: ng serve.
- Open the browser at the Angular local URL and test login, dashboard, and CRUD features.

11.2 Deployment Steps

- Push final frontend, backend, database, report, and demo folders to one public GitHub repository.
- Deploy the frontend on Vercel or Netlify if deployment is required.
- Deploy the ASP.NET Core backend on Azure, Railway, Render, or another supported hosting platform.
- Host the database on Azure SQL, SQL Server hosting, or MongoDB Atlas depending on the selected database.
- Add the live frontend link, backend/API link, and video demo link to README and report.

12. Testing and Validation

Test Case	Input/Action	Expected Result	Status
Signup validation	Submit empty/invalid signup form	Validation messages appear	Pass / To verify
User registration	Enter valid user details	New user created in database	Pass / To verify
Login	Enter valid credentials	JWT token returned and dashboard opens	Pass / To verify
Create project	Add new project record	Project saved and shown in list	Pass / To verify
Create task	Add task under project	Task saved and displayed dynamically	Pass / To verify
Update task status	Change status from pending to completed	Status updates in UI and database	Pass / To verify
Delete record	Delete task/project	Record removed after confirmation	Pass / To verify
Dashboard data	Open dashboard after login	Counts and recent activity are fetched from API	Pass / To verify

13. Requirements Checklist

Requirement	Evidence in Report/Project	Status
Angular + TypeScript frontend	Frontend folder and UI page screenshots	Included
ASP.NET Core Web API backend	Backend folder, controllers, and API documentation	Included
SQL Server or MongoDB database	Database folder with schema and seed scripts; final database screenshot needed	Needs final proof
5-6 dynamic pages	Home, Auth, Dashboard, Main Feature, Detail/Profile, Admin	Included
Reactive forms with validation	Login/signup screenshots and explanation	Included
HTTP Client API connection	API documentation and dashboard data screenshots	Included
Minimum 4 API controllers with CRUD	Auth, Projects, Tasks, Categories, Comments, Users, Dashboard controllers detected	Included
JWT authentication	Auth section and login endpoint	Included
Minimum 5 related tables/collections	Users, Projects, Tasks, Categories, Comments, ActivityLogs	Included
Seed data	seed.sql or database seeder included	Included
Report PDF	This document exported as PDF	Included
Video demo 5-10 minutes	Video demo plan section and link placeholder	To add link
GitHub commits from members	GitHub screenshot placeholder	To verify

14. Video Demo Plan

Video Demo Link: [DEMO VIDEO](#)

15. Conclusion

Task Hub fulfills the main purpose of the Web Technologies semester project by combining a modern Angular frontend, an ASP.NET Core backend, and a relational database design. The project demonstrates dynamic web pages, API integration, authentication, CRUD operations, and database-driven content. Before final submission, the team should replace all placeholders with actual screenshots, add the GitHub repository link, add the video demo link, and ensure that the deployed or local application works end-to-end during demonstration.

Appendix A: Submitted Archive Overview

The project archive contains backend, frontend, database, demo, and related project files. A brief file-structure summary is listed below.

Item	Detected Content
Backend controllers	ApiControllerBase, AuthController, CategoriesController, CommentsController, DashboardController, ProjectsController, TasksController, UsersController
Backend models	ActivityLog, AppUser, Category, Project, ProjectTask, TaskComment
Database files	schema.sql, seed.sql, schema diagram, TaskHub.db
Demo folder	README for demo instructions
Frontend folder	Angular application files and dependencies